

```
# Import necessary libraries
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import mnist

# Step 1: Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Step 2: Combine the training and test data into a single dataset
# Reshape the data into (n_samples, n_features) format
data = np.concatenate((x_train, x_test), axis=0)
labels = np.concatenate((y_train, y_test), axis=0)

# Flatten the images to a 2D array: (n_samples, 28*28)
data = data.reshape(data.shape[0], -1)

# Step 3: Shuffle the dataset
m, n = data.shape
# Stack the data with labels and shuffle them together
data_labels = np.hstack((labels.reshape(m, 1), data))
np.random.shuffle(data_labels)

# Step 4: Split into development and training sets
data_dev = data_labels[:1000].T # First 1000 samples for development
Y_dev = data_dev[0]
X_dev = data_dev[1:n+1] / 255. # Normalize X_dev to the range [0, 1]

data_train = data_labels[1000:].T # Remaining data for training
Y_train = data_train[0]
X_train = data_train[1:n+1] / 255. # Normalize X_train to the range [0, 1]

# Step 5: Get the dimensions of the training set
_, m_train = X_train.shape

# Print the results
print("Shape of X_dev:", X_dev.shape)
print("Shape of Y_dev:", Y_dev.shape)
print("Shape of X_train:", X_train.shape)
print("Shape of Y_train:", Y_train.shape)
print("Number of training samples:", m_train)
```

📄 Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz> 11490434/11490434 ————— 2s 0us/step

```
Shape of X_dev: (784, 1000)
Shape of Y_dev: (1000,)
Shape of X_train: (784, 69000)
Shape of Y_train: (69000,)
Number of training samples: 69000
```

Start coding or [generate](#) with AI.

```

def init_params():
    w1 = np.random.rand(10, 784) - 0.5 # First hidden layer
    # b1 = np.random.rand(10, 1) - 0.5
    # w2 = np.random.rand(10, 10) - 0.5 # Second hidden layer
    # b2 = np.random.rand(10, 1) - 0.5
    # w3 = np.random.rand(10, 10) - 0.5 # Output layer (10 neurons for 10 clas
    # b3 = np.random.rand(10, 1) - 0.5
    # return w1, b1, w2, b2, w3, b3

    # w1 = np.random.randn(10, 784) * np.sqrt(2 / 784) # He initialization
    # b1 = np.zeros((10, 1)) # Initialize biases to 0
    # w2 = np.random.randn(10, 10) * np.sqrt(2 / 10)
    # b2 = np.zeros((10, 1))
    # w3 = np.random.randn(10, 10) * np.sqrt(2 / 10)
    # b3 = np.zeros((10, 1))
    # return w1, b1, w2, b2, w3, b3

    w1 = np.random.randn(128, 784) * np.sqrt(2 / 784) # Increase to 128 neuror
    b1 = np.zeros((128, 1)) # Bias for 128 neurons
    w2 = np.random.randn(64, 128) * np.sqrt(2 / 128) # Second layer with 64 ne
    b2 = np.zeros((64, 1))
    w3 = np.random.randn(10, 64) * np.sqrt(2 / 64) # Output layer remains 10 r
    b3 = np.zeros((10, 1))
    return w1, b1, w2, b2, w3, b3

def ReLU(Z):
    return np.maximum(Z, 0)

def softmax(Z):
    A=np.exp(Z) / sum(np.exp(Z))
    return A

def forward_prop(w1, b1, w2, b2, w3, b3, X):
    # z1 = w1.dot(X) + b1 # Compute the weighted sum of inputs (first hidden
    # a1 = ReLU(z1) # Apply ReLU activation function to first hidden
    # z2 = w2.dot(a1) + b2 # Compute the weighted sum for second hidden layer
    # a2 = ReLU(z2) # Apply ReLU activation function to second hidden
    # z3 = w3.dot(a2) + b3 # Compute the weighted sum for output layer
    # a3 = softmax(z3) # Apply softmax activation function to output layer
    # return z1, a1, z2, a2, z3, a3

    z1 = w1.dot(X) + b1
    a1 = ReLU(z1) # 128 neurons in first hidden layer
    z2 = w2.dot(a1) + b2
    a2 = ReLU(z2) # 64 neurons in second hidden layer
    z3 = w3.dot(a2) + b3
    a3 = softmax(z3) # Output layer (10 neurons for 10 classes)
    return z1, a1, z2, a2, z3, a3

def one_hot(Y):

```

```

    one_hot_Y = np.zeros((Y.size, Y.max() + 1))
    one_hot_Y[np.arange(Y.size), Y] = 1
    one_hot_Y = one_hot_Y.T
    return one_hot_Y

def deriv_ReLU(Z):
    return Z>0

def back_prop(z1, a1, z2, a2, z3, a3, w2, w3, Y, X):
    # OneHot_Y = one_hot(Y)           # Convert labels to one-hot encoding
    # dZ3 = a3 - OneHot_Y             # Compute the error for the output layer
    # dW3 = 1 / m * dZ3.dot(a2.T)     # Gradient for weights of output layer
    # db3 = 1 / m * np.sum(dZ3)       # Gradient for biases of output layer

    # dZ2 = w3.T.dot(dZ3) * deriv_ReLU(z2) # Compute the error for second hidden layer
    # dW2 = 1 / m * dZ2.dot(a1.T)       # Gradient for weights of second hidden layer
    # db2 = 1 / m * np.sum(dZ2)       # Gradient for biases of second hidden layer

    # dZ1 = w2.T.dot(dZ2) * deriv_ReLU(z1) # Compute the error for first hidden layer
    # dW1 = 1 / m * dZ1.dot(X.T)        # Gradient for weights of first hidden layer
    # db1 = 1 / m * np.sum(dZ1)        # Gradient for biases of first hidden layer

    # return dW1, db1, dW2, db2, dW3, db3

    m = X.shape[1]
    OneHot_Y = one_hot(Y)
    dZ3 = a3 - OneHot_Y
    dW3 = 1 / m * dZ3.dot(a2.T)
    db3 = 1 / m * np.sum(dZ3, axis=1, keepdims=True)

    dZ2 = w3.T.dot(dZ3) * deriv_ReLU(z2) # Adjust for 64 neurons
    dW2 = 1 / m * dZ2.dot(a1.T)
    db2 = 1 / m * np.sum(dZ2, axis=1, keepdims=True)

    dZ1 = w2.T.dot(dZ2) * deriv_ReLU(z1) # Adjust for 128 neurons
    dW1 = 1 / m * dZ1.dot(X.T)
    db1 = 1 / m * np.sum(dZ1, axis=1, keepdims=True)

    return dW1, db1, dW2, db2, dW3, db3

def update_params(W1, b1, W2, b2, W3, b3, dW1, db1, dW2, db2, dW3, db3, alpha):
    W1 = W1 - alpha * dW1 # Update weights for the first hidden layer
    b1 = b1 - alpha * db1 # Update biases for the first hidden layer
    W2 = W2 - alpha * dW2 # Update weights for the second hidden layer
    b2 = b2 - alpha * db2 # Update biases for the second hidden layer
    W3 = W3 - alpha * dW3 # Update weights for the output layer
    b3 = b3 - alpha * db3 # Update biases for the output layer
    return W1, b1, W2, b2, W3, b3

def get_predictions(a2):
    return np.argmax(a2,0)

```

```
def get_accuracy(predictions, Y):
    print(predictions, Y)
    return np.sum(predictions == Y)/Y.size

def gradient_descent(X, Y, iterations, alpha):
    w1, b1, w2, b2, w3, b3 = init_params() # Initialize the weights and biases
    for i in range(iterations):
        z1, a1, z2, a2, z3, a3 = forward_prop(w1, b1, w2, b2, w3, b3, X)
        dW1, db1, dW2, db2, dW3, db3 = back_prop(z1, a1, z2, a2, z3, a3, w2, w3, w1, b1, w2, b2, w3, b3)
        w1, b1, w2, b2, w3, b3 = update_params(w1, b1, w2, b2, w3, b3, dW1, db1, dW2, db2, dW3, db3)
        if i % 10 == 0:
            print("iterations: ", i)
            print("accuracy: ", get_accuracy(get_predictions(a3), Y))
    return w1, b1, w2, b2, w3, b3
```

```
w1, b1, w2, b2, w3, b3 = gradient_descent(X_train, Y_train, 500, 0.05)
```

```
iterations: 0
[8 9 3 ... 1 3 4] [7 4 3 ... 8 0 2]
accuracy: 0.09485507246376812
iterations: 10
[8 7 3 ... 1 6 9] [7 4 3 ... 8 0 2]
accuracy: 0.38308695652173913
iterations: 20
[7 4 7 ... 2 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.637695652173913
iterations: 30
[7 4 7 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.7237971014492753
iterations: 40
[7 4 7 ... 8 0 1] [7 4 3 ... 8 0 2]
accuracy: 0.7687391304347826
iterations: 50
[7 4 7 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.7954782608695652
iterations: 60
[7 4 7 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8147536231884058
iterations: 70
[7 4 7 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8293768115942028
iterations: 80
[7 4 7 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8406811594202899
iterations: 90
[7 4 7 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8507391304347826
iterations: 100
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.858304347826087
iterations: 110
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8608157894736842
```

```

[1 4 3 ... 8 0 8] [1 4 3 ... 8 0 2]
accuracy: 0.865
iterations: 120
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.870072463768116
iterations: 130
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8746231884057971
iterations: 140
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8781014492753623
iterations: 150
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8814637681159421
iterations: 160
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8843768115942029
iterations: 170
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8867826086956522
iterations: 180
[7 4 3 ... 8 0 8] [7 4 3 ... 8 0 2]
accuracy: 0.8892028985507247
iterations: 190

```

```

def make_predictions(X, W1, b1, W2, b2, W3, b3):
    # Perform forward propagation through all 3 layers
    _, _, _, A2, _, A3 = forward_prop(W1, b1, W2, b2, W3, b3, X)

    # Use the output layer (A3) to get predictions
    predictions = get_predictions(A3)

    return predictions

```

```

import matplotlib.pyplot as plt

def test_prediction(index, W1, b1, W2, b2, W3, b3):
    # Extract the image from the dataset at the given index
    current_image = X_train[:, index, None] # Shape should be (784, 1) for a s

    # Make predictions using the 3-layer model (now passing all weights and bias)
    prediction = make_predictions(current_image, W1, b1, W2, b2, W3, b3)

    # Get the true label from Y_train
    label = Y_train[index]

    # Print the predicted and actual label
    print(f"Prediction: {prediction}")
    print(f"Label: {label}")

    # Reshape and scale the image to 28x28 for visualization (MNIST format)
    current_image = current_image.reshape((28, 28)) * 255 # Reshape to 28x28 a

    # Plot the image
    plt.imshow(

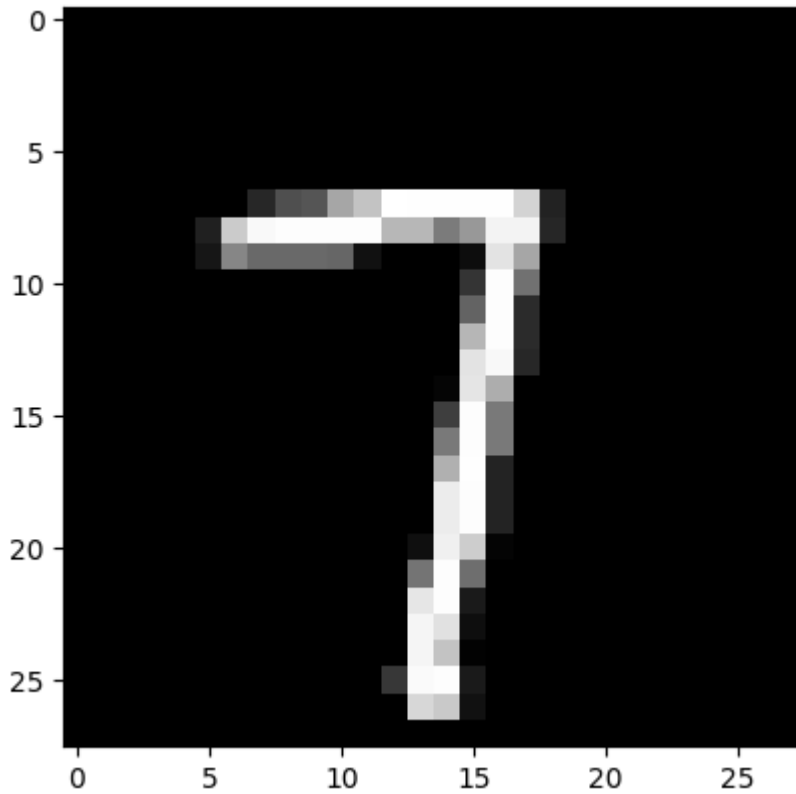
```

```
plt.gray()  
plt.imshow(current_image, interpolation='nearest')  
plt.show()
```

```
test_prediction(0, w1, b1, w2, b2, w3, b3)  
test_prediction(1, w1, b1, w2, b2, w3, b3)  
test_prediction(2, w1, b1, w2, b2, w3, b3)  
test_prediction(3, w1, b1, w2, b2, w3, b3)
```

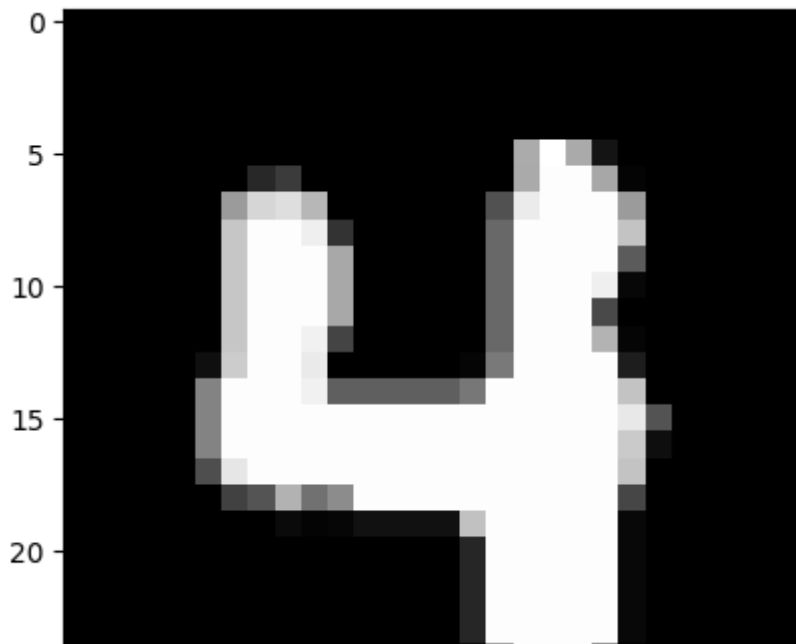
Prediction: [7]

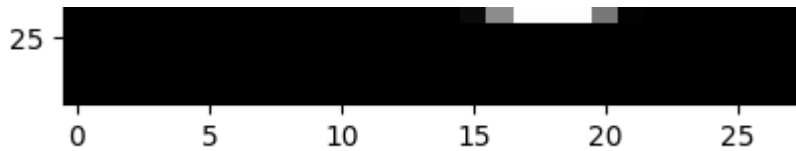
Label: 7



Prediction: [4]

Label: 4





Prediction: [3]

Label: 3



```
# Use the three-layer network for making predictions on the development set
dev_predictions = make_predictions(X_dev, w1, b1, w2, b2, w3, b3)
```

```
# Get the accuracy of the predictions
accuracy = get_accuracy(dev_predictions, Y_dev)
```

```
# Print the accuracy
print(f"Development Set Accuracy: {accuracy}")
```

```
[5 4 4 9 9 4 1 9 2 3 5 8 4 6 1 9 6 7 9 3 9 4 0 3 1 2 2 8 8 6 5 3 8 4 1 7 9
 8 1 8 2 0 2 6 0 8 2 4 8 3 1 4 5 9 1 6 7 9 3 3 5 7 3 4 0 8 5 9 0 1 8 0 2 5
 4 3 1 0 1 7 5 7 6 7 0 3 3 0 2 7 6 7 8 4 2 8 2 1 7 1 4 1 0 9 7 8 7 0 6 1 2
 3 3 3 3 5 8 2 0 5 9 6 6 1 3 3 6 6 9 8 2 5 2 1 7 2 2 5 5 0 9 1 8 0 8 2 8 3
 4 3 8 8 3 2 9 2 0 1 6 5 6 8 8 8 0 2 1 3 7 3 6 1 6 4 6 8 5 6 5 5 9 0 7 9 1
 6 0 1 8 6 4 2 8 3 4 5 9 3 4 8 0 4 6 4 4 4 1 0 8 5 1 3 2 7 6 8 8 2 2 0 4 4
 6 7 8 8 2 2 1 7 6 6 5 0 9 7 0 3 9 0 2 7 8 7 4 2 5 5 0 9 2 4 7 7 6 8 7 7 2
 7 7 4 0 0 2 6 4 0 0 3 5 1 2 5 8 4 0 5 6 2 6 8 0 8 3 4 5 5 1 9 5 9 0 3 0 1
 6 9 3 8 3 6 0 9 6 5 8 1 7 9 5 1 8 5 9 1 9 3 5 2 6 6 7 7 6 8 2 2 7 9 6 0 9
 1 4 1 7 4 8 1 2 2 3 4 7 2 5 4 8 1 6 1 6 0 4 6 7 8 2 5 7 7 9 9 0 8 1 7 7 2
 7 4 1 6 9 1 5 5 2 5 8 1 3 9 8 0 5 7 7 9 6 1 8 4 6 7 7 3 1 4 5 4 4 9 8 2 8
 5 9 2 1 5 2 6 7 0 4 7 1 5 1 9 1 0 0 3 4 4 7 0 3 0 4 2 8 8 1 7 4 7 0 4 3 6
 4 5 3 9 3 9 1 3 9 1 9 2 3 6 3 3 4 5 4 2 1 8 1 0 7 4 4 9 3 2 6 1 2 6 1 8 9
 9 0 1 2 0 7 1 0 9 8 6 6 8 6 5 9 0 5 1 1 8 2 7 2 4 3 6 8 3 4 1 0 1 1 7 3 5
 8 7 8 0 0 9 8 0 7 0 7 3 0 4 3 7 0 5 3 4 9 6 3 7 2 4 3 6 8 0 9 0 4 6 2 5 8
 3 2 2 3 3 7 1 5 0 8 6 2 3 4 7 4 7 8 6 4 0 6 5 5 0 8 0 1 9 8 7 3 6 0 8 6 3
 5 4 2 1 4 8 5 1 6 2 3 4 7 8 3 6 4 1 9 9 7 6 9 2 9 3 8 1 3 1 1 6 3 0 1 4 7
 5 3 8 5 7 3 9 2 0 4 6 2 2 0 6 2 0 3 3 0 0 9 7 7 0 5 3 4 4 9 1 8 2 6 3 4 2
 4 8 9 0 1 1 6 4 5 1 7 3 5 0 8 4 7 0 4 3 0 1 7 8 0 0 7 7 4 7 3 5 1 1 0 3 1
 8 3 0 3 0 1 9 5 2 9 5 0 0 0 2 3 8 5 3 3 2 7 7 5 9 8 6 9 9 9 2 0 0 6 0 3 4
 8 1 5 3 6 5 0 9 8 9 1 9 1 4 2 7 5 9 9 7 2 5 5 4 2 2 1 4 0 5 9 9 0 2 3 1 7
 9 7 5 2 1 2 2 5 9 5 3 0 3 1 3 6 1 4 7 1 1 0 6 9 9 6 7 1 9 3 6 3 7 2 4 5 9
 4 9 7 3 4 1 3 2 8 1 6 0 1 3 5 3 8 0 2 4 8 2 6 5 0 0 9 7 4 5 3 1 8 0 1 7 5
 1 5 3 3 5 1 9 7 5 8 7 5 0 9 5 9 9 4 3 7 7 9 3 3 7 7 6 7 2 4 4 8 0 4 2 7 6
 5 9 6 2 2 5 7 0 8 6 5 2 2 4 0 9 2 7 8 0 5 4 9 0 5 0 7 0 9 6 1 4 8 6 3 7 2
 8 3 4 0 1 9 8 1 9 5 2 7 4 4 1 9 7 7 8 1 3 1 4 7 8 8 3 6 4 8 1 8 3 6 4 7 8
 7 2 8 8 2 9 5 2 0 0 5 3 3 0 5 1 9 3 7 9 3 1 4 2 9 9 5 1 7 1 6 7 8 8 4 1 7
 0] [5 4 4 9 9 4 1 9 2 3 5 8 4 0 1 9 8 7 3 3 3 4 0 3 1 2 4 8 8 6 5 3 8 4 1
 8 1 8 2 5 2 6 0 8 2 4 8 3 1 4 5 4 1 6 7 9 3 3 5 7 3 4 0 8 5 9 0 1 8 0 2 5
 9 3 1 0 1 7 5 7 0 7 0 3 3 0 2 7 6 7 8 4 2 8 2 1 7 1 4 1 0 9 7 8 7 0 6 1 2
 1 3 3 3 6 8 8 0 5 9 6 6 1 3 3 6 6 9 8 2 5 2 1 7 2 2 5 5 0 9 1 3 0 8 2 8 3
 4 3 8 8 3 2 9 2 0 1 6 5 6 8 8 8 0 2 1 3 7 3 6 1 5 4 6 8 5 6 5 5 9 0 7 9 1
 0 0 1 4 6 4 2 8 3 9 5 8 3 4 8 5 4 6 4 4 6 1 0 8 5 1 3 2 7 6 8 8 2 2 0 4 4
 6 3 0 2 2 2 1 7 6 6 5 0 9 7 0 3 9 0 2 4 8 7 2 2 5 5 0 9 2 4 7 7 6 8 7 7 2
 7 7 4 0 2 2 6 4 0 0 3 5 1 2 0 8 2 0 5 6 2 6 8 0 8 3 4 5 5 1 9 5 2 2 3 0 1
 6 9 3 8 3 6 0 9 6 5 8 1 7 9 5 1 8 5 9 1 9 3 5 7 6 6 9 7 6 8 8 2 7 9 6 0 9
 1 4 1 7 4 8 1 2 2 3 4 7 2 5 4 8 1 6 1 6 0 4 6 7 8 2 5 7 7 0 0 0 8 1 7 7 2
```

Development Set Accuracy: 0.904